A PROJECT REPORT ON

FILE ENCRYPTION TOOL

For the Course

"Project-I"

Course Code: ICT-1200

SUBMITTED BY

Md. Fahim Foysal
Student ID: IT-25055
Session: 2024–2025

SUPERVISED BY

Dr. Ziaur Rahman
Professor
Department of ICT, MBSTU

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

(MBSTU)



ACKNOWLEDGMENTS

I would like to express my sincere gratitude to everyone who supported me during the development of the File Encryption Tool project. This project gave me an opportunity to apply C++ programming, file handling, graphical user interface design, and basic cryptographic concepts in a practical desktop application.

First, I would like to express my respectful thanks to my supervisor, Dr. Ziaur Rahman, Professor, Department of ICT, Mawlana Bhashani Science and Technology University (MBSTU), for providing academic guidance and encouragement for this project. His supervision helped me keep the work focused on the requirements of the course.

I am also grateful to the Department of Information and Communication Technology for the learning environment and the programming knowledge that supported this work. The concepts learned in C++ and software development were important for designing the application and testing its features.

I would like to acknowledge the developers and documentation teams behind Qt, OpenSSL, and the C++ tools used during development. Their publicly available documentation helped me understand GUI development, file operations, password-based key derivation, and AES encryption interfaces.

Finally, I thank my family, classmates, and everyone who encouraged me during the project. Any remaining errors or limitations are my responsibility.

Md. Fahim Foysal
Department of ICT, MBSTU

TABLE OF CONTENTS

ACKNOWLEDGMENTS 2
ABSTRACT 4
CHAPTER 1. INTRODUCTION 5
1.1 Background and Context 5
1.2 Research Problem 5
1.3 Significance of Study 5
1.4 Research Question 6
1.5 Objectives 6
1.6 Project Statement 6
CHAPTER 2. LITERATURE REVIEW 7
2.1 Theoretical Framework 7
2.2 Current State of Knowledge 7
2.3 Research Gap 8
2.4 Key Concepts and Definitions 8
CHAPTER 3. METHODOLOGY 9
3.1 Research Design 9
3.2 Data Collection Methods 9
3.3 Sampling Strategy 9
3.4 Analytical Approach 10
3.5 Ethical Considerations 10
CHAPTER 4. RESULTS AND ANALYSIS 11
4.1 Data Representation 11
4.1.1 Application Interface and File Selection 11
4.1.2 Caesar and Vigenère Modes 11
4.1.3 AES-128 Mode 11
4.1.4 Password Verification and File Lifecycle 12
4.2 Relation to Research Questions 12
4.3 Implications of Result 12
4.4 Limitations of Study 12
CHAPTER 5. DISCUSSION 13
5.1 Synthesis of Findings 13
5.2 Relation to Research Questions 13
5.3 Implications of Results 13
5.4 Limitations of Study 13
CHAPTER 6. CONCLUSION AND RECOMMENDATIONS 14
6.1 Summary of Key Points 14
6.2 Contributions to Field 14
6.3 Recommendations 14
6.4 Future Research Directions 14
REFERENCES 15
APPENDICES 16

ABSTRACT

File Encryption Tool is a Windows desktop application developed in C++ with the Qt framework. The project demonstrates how a graphical application can allow a user to select a file, enter a password, choose an encryption algorithm, and encrypt or decrypt the selected file locally. The project is designed as an academic software development exercise focused on file handling, GUI programming, and introductory applied cryptography.

The application provides three algorithm choices: Caesar Cipher, Vigenère Cipher, and AES-128. Caesar and Vigenère are classical ciphers included for learning and comparison. AES-128 is the modern symmetric encryption option and is implemented using the OpenSSL cryptographic library. For the AES mode, the application generates a random salt and a random initialization vector (IV), derives a 128-bit key from the password using PBKDF2-HMAC-SHA256, and uses AES-128-CBC for encryption and decryption. The implementation also stores a SHA-256 password-verification value in the encrypted file format so that the application can check the entered password before continuing.

The interface includes file browsing, password entry, a Show Password option, algorithm selection, and status messages. The application handles both text and binary file data through byte arrays and file I/O. As designed, successful encryption removes the original file after the encrypted output is created, while successful decryption removes the encrypted input after the decrypted output is created. Incorrect password handling was tested and displays an error message rather than continuing with decryption.

The project was developed using Qt Creator on Windows and built as a local desktop application. The project source includes CMake configuration and the Qt application files. Testing focused on file selection, password visibility, algorithm selection, encryption/decryption success, wrong-password behavior, and the expected file lifecycle. The project demonstrates useful concepts, but Caesar and Vigenère are not suitable for protecting sensitive information, and the AES implementation would benefit from authenticated encryption and additional format validation in future work.

Keywords: C++, Qt, file handling, encryption, decryption, AES-128, OpenSSL, PBKDF2, SHA-256.

CHAPTER 1. INTRODUCTION

1.1 Background and Context

Digital files are used for academic, personal, and professional work. Some files may contain private information, and users may want to make their contents unreadable to people who do not know the correct password. Encryption is one method used to transform readable data into a form that can be recovered only through the matching decryption process and required key.

File encryption tools are available in many forms, but building a small desktop application is a useful way to understand how file handling and cryptographic operations work together. This project uses C++ for program logic and Qt for the graphical user interface. It runs locally on a Windows computer rather than requiring a web server.

1.2 Research Problem

The main problem addressed by this project is how to provide a simple interface for selecting a file and applying encryption or decryption using a password. A command-line workflow may be difficult for a beginner to use, while a graphical interface can make the main steps more visible. A second challenge is ensuring that the application treats file data consistently and reports common errors, including an incorrect password.

1.3 Significance of Study

The project has academic value because it connects programming topics with a practical application. It demonstrates file input/output, byte-oriented data processing, GUI events, external library integration, and basic security concepts. It also provides a concrete example for discussing the differences between classical educational ciphers and a modern symmetric algorithm.

The project should not be understood as a fully audited security product. Its value is primarily educational: it illustrates the flow of a local file-encryption application and provides a basis for further improvements.

1.4 Research Question

The project is guided by the following questions:

- How can a Qt-based C++ desktop interface make file encryption and decryption straightforward for a local user?
- How can one application demonstrate classical ciphers and a modern AES-128 encryption option?
- How can password verification and password-based key derivation be integrated into the file-processing workflow?
- How can the application handle incorrect passwords and manage input/output files after successful operations?

1.5 Objectives

The main objective is to develop a local Windows desktop application that can encrypt and decrypt selected files using a password and a chosen algorithm.

Specific objectives are:

- Create a graphical interface using Qt Widgets.
- Allow a user to browse for a file and enter a password.
- Implement Caesar Cipher and Vigenère Cipher as educational options.
- Integrate OpenSSL for AES-128-CBC encryption and decryption.
- Use SHA-256 for a stored password-verification value and PBKDF2-HMAC-SHA256 to derive the AES key from the password and salt.
- Generate random salt and IV values for the AES workflow.
- Display status and error messages for important operations.
- Test the main user workflows, including incorrect-password behavior.

1.6 Project Statement

File Encryption Tool is a C++/Qt Windows desktop application that allows a user to choose a file, select Caesar, Vigenère, or AES-128, provide a password, and perform encryption or decryption. It demonstrates a local file-processing workflow and applies OpenSSL-based AES-128-CBC operations with password-based key derivation. The application is intended for learning and demonstration; further security review would be needed before relying on it for high-value or highly sensitive data.

CHAPTER 2. LITERATURE REVIEW

2.1 Theoretical Framework

The project combines ideas from file systems, graphical user interfaces, and cryptography. File input/output reads bytes from a selected path and writes processed bytes to an output path. Qt Widgets provides controls such as buttons, text fields, a combo box, a checkbox, and status labels. Signals and slots connect user actions to the corresponding application logic.

Cryptography transforms data using an algorithm and a key. Symmetric encryption uses the same secret key for encryption and decryption. In AES-128, the key length is 128 bits. The project uses AES in Cipher Block Chaining (CBC) mode through OpenSSL. CBC requires an initialization vector and padding for data that does not exactly match the block size. The IV is not a password and does not replace the secret key; it is stored with the encrypted data so the same IV can be used during decryption.

A password is usually not a strong, fixed-length encryption key by itself. PBKDF2-HMAC-SHA256 applies a password-based derivation process using a salt and an iteration count to produce key bytes. In this project, the documented implementation uses 100,000 iterations and derives 16 bytes for AES-128. A random salt helps ensure that the same password does not always produce the same derived key across different encryption operations.

The implementation also stores a SHA-256 hash of the password for verification. A hash is a one-way digest, not encryption. This verification value helps the program reject an incorrect password before attempting the rest of decryption, but it should not be confused with a complete authenticated-encryption design.

2.2 Current State of Knowledge

Modern software libraries provide established implementations of standard cryptographic algorithms. Using a maintained library such as OpenSSL is generally preferable to writing a modern block cipher from scratch. The Qt framework supports cross-platform desktop interfaces and file-dialog interactions, while CMake provides a build configuration that can locate Qt and OpenSSL dependencies.

Classical ciphers such as Caesar and Vigenère are historically important and useful for learning how key-dependent transformations work. However, their limited key spaces and structural weaknesses make them unsuitable for modern confidentiality requirements. Their inclusion in this project is for demonstration and comparison, not as a recommendation for real-world file security.

2.3 Research Gap

A small educational program can demonstrate individual algorithms, but it is more instructive to combine algorithm selection, GUI events, file I/O, password checks, and error reporting in one workflow. The project addresses this learning gap by presenting the process through a single desktop interface.

The project does not attempt to introduce a new cryptographic algorithm or claim that its implementation is formally secure. Instead, it focuses on integration and usability at an introductory level. Important areas remain for future work, including authenticated encryption, stronger file-format validation, safer deletion policies, and broader automated testing.

2.4 Key Concepts and Definitions

Encryption: A process that transforms readable data into ciphertext using an algorithm and a key.

Decryption: The reverse operation intended to recover the original data from ciphertext with the correct key.

Ciphertext: The transformed output produced by encryption.

Symmetric encryption: Encryption in which the same secret key is used to encrypt and decrypt.

AES-128: The Advanced Encryption Standard with a 128-bit key.

CBC mode: Cipher Block Chaining, a block-cipher mode that combines each plaintext block with the previous ciphertext block; the first block uses an IV.

Initialization vector (IV): A value used by a cipher mode to initialize encryption. It is not a substitute for the secret key.

Salt: A random value used in password-based key derivation to make derived keys differ across encryption operations.

PBKDF2-HMAC-SHA256: A password-based key derivation function that uses HMAC-SHA256 and repeated work to derive key material.

SHA-256: A cryptographic hash function that produces a 256-bit digest.

OpenSSL: A cryptographic software library used by the project for AES and random-byte generation.

Qt Widgets: A Qt module used to build the application's graphical interface.

Binary file handling: Reading and writing file contents as bytes rather than assuming they are plain text.

CHAPTER 3. METHODOLOGY

The project followed a practical, iterative software-development approach. The application was built, run locally, and tested in stages so that interface behavior and algorithm behavior could be checked separately.

3.1 Research Design

- Requirement identification: define the local desktop use case, the available algorithms, password input, file browsing, and expected output behavior.
- Interface construction: create the Qt Widgets interface and connect the controls to C++ event handlers.
- File-processing implementation: read the selected file as bytes, apply the selected algorithm, and write the resulting bytes to an output file.
- Cryptographic integration: use OpenSSL for AES-128-CBC, random-byte generation, and PBKDF2-based key derivation.
- Verification: run manual tests for successful encryption/decryption and incorrect-password handling.

3.2 Data Collection Methods

No survey dataset or human-subject experiment was used. Development evidence came from the project requirements, code inspection, compiler/build output, and manual operation of the application. Test files were used to observe whether encryption created an output file and whether decryption recovered the data.

Because this is a software implementation project, the relevant evidence is functional behavior rather than statistical measurements from a sample of participants. The report therefore describes observed application behavior and does not claim measured performance or user satisfaction.

3.3 Sampling Strategy

The project used purposive functional test cases rather than statistical sampling. The selected cases represent core workflows: selecting a file, choosing an algorithm, entering a password, encrypting, decrypting, and entering an incorrect password. Both text and binary file support are part of the intended file-processing design; a comprehensive test matrix across many file formats and file sizes remains a possible extension.

3.4 Analytical Approach

The main analysis was a functional check of each workflow against its expected result. For a successful encryption, the expected result is an encrypted output file and a success status. Under the project's chosen file-lifecycle behavior, the original file is removed after the encrypted output has been created successfully. For successful decryption, the expected result is a decrypted output and a success status, followed by removal of the encrypted input. Incorrect-password testing checks that the application reports the problem and does not proceed as if decryption succeeded.

The AES workflow is also reviewed at the design level: a random salt and IV are generated; PBKDF2-HMAC-SHA256 derives a 16-byte key from the password and salt using 100,000 iterations; AES-128-CBC performs the data transformation; and a SHA-256 password-verification value is used to check the password. These design details describe the implementation, not a formal security audit.

3.5 Ethical Considerations

The application operates on files chosen by the user. Since the current design removes the input file after successful processing, users should understand the file-lifecycle behavior before using it with important data. For real-world use, a safer default would be to retain the original file or request explicit confirmation, and to recommend backups.

Passwords should not be shared or embedded in source code. The report does not include any personal files or passwords. The project is presented as an educational demonstration, and the limitations of classical ciphers and CBC-mode authentication are stated to avoid overstating its security.

Development Environment

The project was developed on Windows 11 using Qt Creator with Qt 6.11.1 and a MinGW 64-bit kit. The build system is CMake and the source uses C++17. OpenSSL was integrated through the MSYS2 UCRT64 installation, with CMake configured to locate that installation. A Windows release build was tested locally.

Main Project Files

- main.cpp — application entry point.
- mainwindow.h — main window class declaration.
- mainwindow.cpp — interface event handlers and file-processing logic.
- mainwindow.ui — Qt Designer interface description.
- CMakeLists.txt — project build configuration and dependency discovery.

CHAPTER 4. RESULTS AND ANALYSIS

4.1 Data Representation

The application brings the main operations into one desktop window. The user selects a file, chooses an algorithm, enters a password, and starts encryption or decryption. Status messages communicate selection and operation results. The following subsections describe the implemented components and the observed tests.

4.1.1 Application Interface and File Selection

The main window is titled "File Encryption Tool." It includes a file path field, a Browse button, a password field, a Show Password checkbox, an algorithm selector, encryption/decryption controls, and status information. The Browse action opens a file-selection dialog and places the chosen path into the path field. A status message confirms that a file has been selected.

The password field uses password echo mode by default. The Show Password checkbox allows the user to view the entered password when needed. The status area is used for messages such as readiness, successful encryption, and error conditions.

4.1.2 Caesar and Vigenère Modes

Caesar Cipher and Vigenère Cipher are included as classical educational options. In the Caesar implementation, the password is used to calculate a byte shift by summing the password characters' Unicode values and taking the result modulo 256. The same shift is reversed during decryption.

In the Vigenère implementation, each input byte is shifted using the corresponding password character's Unicode value modulo 256. The key index repeats across the input bytes. The encrypt and decrypt functions operate on byte arrays, allowing the same general processing path to be used for file data. These methods are simple demonstrations and do not provide modern cryptographic security.

4.1.3 AES-128 Mode

The AES-128 option uses OpenSSL. The application generates a 16-byte random salt and a 16-byte random IV. It derives a 16-byte AES key from the password through PBKDF2-HMAC-SHA256 with 100,000 iterations. The data is encrypted using AES-128-CBC. During decryption, the application reads the stored values, derives the key again from the entered password, and attempts to decrypt the ciphertext.

The file format also includes a SHA-256 hash used for password verification. If the entered password does not match the stored verification value, the application reports that the password is incorrect. The verification hash is a separate check and should not be described as an authentication tag for the ciphertext.

4.1.4 Password Verification and File Lifecycle

Testing confirmed that an incorrect password produces the message "The password is incorrect." This behavior prevents the application from treating a wrong password as a successful operation. The application also displays status messages for file selection and successful encryption.

The selected file-lifecycle rule is intentionally destructive: after successful encryption, the original file is removed and the encrypted output is kept; after successful decryption, the encrypted input is removed and the decrypted output is kept. This rule was tested for the Caesar and Vigenère workflows, and AES functionality was also tested. Because deletion can cause data loss if an output is wrong or the user selected the wrong file, the rule should be clearly understood and future versions should consider a safer option.

4.2 Relation to Research Questions

The interface addresses the first question by bringing browsing, password entry, algorithm choice, and status messages into one local window. The three algorithm choices address the second question by presenting classical educational ciphers beside AES-128. The PBKDF2, salt, IV, and password-verification steps address the third question in the AES workflow. Manual tests of successful operations and incorrect-password handling address the fourth question at a functional level.

4.3 Implications of Result

The results show that a small Qt application can connect GUI events, file I/O, and external cryptographic library calls in one workflow. The project is useful for learning how an application passes user input to processing functions and reports outcomes. It also demonstrates why password handling, random values, output naming, and file deletion policies are important parts of an encryption tool, not merely interface details.

4.4 Limitations of Study

- Caesar and Vigenère are educational ciphers and should not be used to protect confidential files.
- AES-CBC does not by itself authenticate ciphertext. A modern design should use an authenticated-encryption mode such as AES-GCM or add a carefully designed authentication mechanism.
- The SHA-256 password-verification value is not a ciphertext authentication tag.
- The report is based on manual functional testing; it does not claim exhaustive tests for every file type, file size, or failure condition.
- Deleting source files automatically can cause data loss. A safer workflow should retain the source by default or provide explicit confirmation.
- The application is documented and tested as a Windows desktop build; other operating systems require their own compatible build and dependency setup.

CHAPTER 5. DISCUSSION

The File Encryption Tool project demonstrates a practical connection between C++ programming, Qt GUI development, file operations, and cryptographic library integration. The results are best understood as evidence that the core workflows operate in the tested local environment, rather than as proof of production-grade security.

5.1 Synthesis of Findings

1. GUI usability: the Browse control, password field, algorithm selector, and status labels provide a clear sequence for the main user tasks.
2. Algorithm comparison: Caesar and Vigenère illustrate simple password-dependent transformations, while AES-128 demonstrates the use of a standard cryptographic library.
3. Password-based AES setup: salt-based PBKDF2 key derivation and a random IV are used to avoid relying on the password as a raw AES key.
4. Error feedback: wrong-password handling gives the user a clear message instead of silently reporting success.
5. File lifecycle: automatic input deletion matches the selected project behavior, but it creates a usability and safety trade-off.

5.2 Relation to Research Questions

The research questions are answered at a functional demonstration level. A single Qt window can make the steps easy to follow; several algorithms can be integrated into one application; and the AES mode can derive a key from a password using PBKDF2. Testing also shows that incorrect-password feedback can be incorporated into the decryption workflow. The project does not establish that the design is secure against all attacks, nor does it provide a comparative performance benchmark.

5.3 Implications of Results

For students learning software development, the project provides an example of decomposing an application into interface controls, event handlers, file-processing routines, and library calls. It highlights that security-sensitive programs require more than a working encryption function: the file format, password derivation, integrity checks, error handling, and user expectations all affect reliability.

The project can be extended in a controlled way. A future version could introduce authenticated encryption, a versioned file header, stronger validation of malformed files, a safer output policy, and automated tests. Such changes should be documented and tested rather than assumed to be present in the current version.

5.4 Limitations of Study

The evaluation was limited to local manual tests and the current Windows build. It did not include formal cryptographic review, penetration testing, large-scale performance testing, or usability testing with a representative group of users. The classical algorithms remain intentionally weak, and the AES-CBC file format would need integrity protection before it could be recommended for sensitive real-world use.

CHAPTER 6. CONCLUSION AND RECOMMENDATIONS

6.1 Summary of Key Points

File Encryption Tool is a C++ desktop application built with Qt Widgets. It supports file selection, password entry, three algorithm choices, and status reporting. Caesar and Vigenère provide educational examples, while AES-128-CBC is implemented through OpenSSL. In the AES workflow, the program uses a random salt, a random IV, PBKDF2-HMAC-SHA256 with 100,000 iterations, a 16-byte derived key, and a SHA-256 password-verification value.

The main workflows were exercised locally, including successful encryption/decryption and incorrect-password behavior. The project demonstrates the integration of interface design, file handling, and cryptographic library functions. It is an academic prototype and should not be presented as a fully audited secure storage product.

6.2 Contributions to Field

- Practical learning: connects C++ file handling and Qt GUI events in a single desktop application.
- Cryptography demonstration: shows the difference between classical ciphers and AES-based symmetric encryption.
- Library integration: demonstrates using OpenSSL from a CMake-based Qt project.
- Security awareness: makes password handling, salt, IV, key derivation, and file-lifecycle decisions visible for discussion.

6.3 Recommendations

6. Prefer an authenticated encryption mode such as AES-GCM for new versions, with correct nonce handling and tag verification.
7. Do not use Caesar or Vigenère to protect real confidential data; label them as educational modes.
8. Consider keeping the original file by default, or ask for explicit confirmation before deleting any file.
9. Validate file headers, lengths, algorithm identifiers, and malformed input before processing.
10. Add automated tests for empty files, large files, binary files, corrupted encrypted data, wrong passwords, and file permission errors.
11. Document the encrypted file format and provide clear recovery guidance to prevent accidental data loss.

6.4 Future Research Directions

Future work could compare authenticated-encryption designs, add an explicit file-format version, and separate encryption metadata from ciphertext in a structured container. The application could also be packaged for other operating systems after testing compatible Qt and OpenSSL builds. Further work could add progress reporting for large files, a non-destructive output mode, automated regression tests, and a usability evaluation with student volunteers. Any security improvement should be reviewed carefully and tested before being described as complete.

REFERENCES

The following official documentation and standards provide background for the technologies and concepts used in this project. They are listed as technical references; the report does not claim that the project has undergone an external security audit.

1. The Qt Company. Qt Documentation: Qt Widgets. https://doc.qt.io/qt-6/qtwidgets-index.html
2. The Qt Company. Qt Documentation: QFileDialog. https://doc.qt.io/qt-6/qfiledialog.html
3. OpenSSL Project. OpenSSL Documentation: EVP Symmetric Encryption and Decryption. https://docs.openssl.org/
4. OpenSSL Project. OpenSSL Documentation: RAND_bytes. https://docs.openssl.org/
5. OpenSSL Project. OpenSSL Documentation: PKCS5_PBKDF2_HMAC. https://docs.openssl.org/
6. National Institute of Standards and Technology (NIST). FIPS PUB 197: Advanced Encryption Standard (AES). https://csrc.nist.gov/pubs/fips/197/final
7. National Institute of Standards and Technology (NIST). SP 800-38A: Recommendation for Block Cipher Modes of Operation. https://csrc.nist.gov/pubs/sp/800/38/a/final
8. National Institute of Standards and Technology (NIST). SP 800-132: Recommendation for Password-Based Key Derivation. https://csrc.nist.gov/pubs/sp/800/132/final
9. CMake. CMake Documentation. https://cmake.org/documentation/
10. File Encryption Tool. Project source repository. https://github.com/fahim-foysal828/FileEncryptionTool

APPENDICES

A.1 Source Code Listing

The project source is organized into the following files:

- main.cpp — starts the Qt application and opens the main window.
- mainwindow.h — declares the main window class and its interface members.
- mainwindow.cpp — contains the main window event handlers, file operations, classical-cipher functions, and AES/OpenSSL workflow.
- mainwindow.ui — defines the visual layout created with Qt Designer.
- CMakeLists.txt — defines the C++ standard, Qt Widgets dependency, OpenSSL discovery, source files, and build target.

The complete source code is maintained in the project's GitHub repository: https://github.com/fahim-foysal828/FileEncryptionTool. The following short excerpts summarize key implementation ideas; they are explanatory excerpts, not a replacement for the full source files.

Representative AES key derivation call

```
PKCS5_PBKDF2_HMAC(passwordBytes, passwordLength,
    salt, saltLength, 100000, EVP_sha256(), 16, keyBytes);
```

A.2 Feature Documentation

- File selection: the Browse button opens a file dialog and fills the selected path field.
- Password display: the password field is hidden by default; the Show Password checkbox toggles visibility.
- Algorithm selection: the user chooses Caesar, Vigenère, or AES-128.
- Encryption and decryption: the application processes file contents and writes the output file.
- Password verification: a SHA-256 value is checked to reject an incorrect password in the implemented file format.
- AES setup: a random salt and IV are used; PBKDF2-HMAC-SHA256 derives a 16-byte key with 100,000 iterations.
- Status messages: the interface communicates file selection, success, and relevant error states.
- File lifecycle: the chosen behavior removes the input file after successful encryption/decryption output is created.

A.3 Testing & Deployment Documentation

Test	Input / Action	Expected / Observed Result	Status
Browse file	Choose an existing file	Selected path appears; status indicates file selected	Tested
Show Password	Toggle checkbox	Password visibility changes	Tested
Caesar workflow	Encrypt then decrypt with matching password	Output generated and workflow completes	Tested
Vigenère workflow	Encrypt then decrypt with matching password	Output generated and workflow completes	Tested
AES-128 workflow	Encrypt/decrypt with matching password	AES workflow exercised locally	Tested
Incorrect password	Decrypt with wrong password	"The password is incorrect" message appears	Tested
Release build	Build Release configuration and run locally	Standalone Windows release run tested	Tested

Deployment note: the Windows Release build was created and run locally. The repository includes the CMake project files. Rebuilding on another computer requires a compatible Qt kit, a C++ compiler, CMake, and OpenSSL development libraries. Other platforms may be possible because Qt and OpenSSL are available on multiple platforms, but this report does not claim that a macOS or Linux build has been tested.